

ADVANCED DATA SCIENCE & AI

GRADUATION PROJECT OPTIONS

Roadmap Area	How the graduation project must demonstrate it
Python / Math / Data Prep	Reusable Python modules, NumPy/Pandas, statistics, preprocessing, EDA, feature engineering.
SQL & Data Pipelines	Relational schema + analytical SQL + joins/window functions; reproducible ETL/ELT pipeline.
Machine Learning	Baseline models, model comparison, cross-validation, tuning, ROC/PR analysis, error analysis.
Deep Learning	A meaningful neural-network upgrade: MLP/CNN/LSTM/Transformer/Autoencoder depending on project.
Cloud / Big Data	Spark processing or scalable pipeline where appropriate; cloud-ready data/model artifacts.
Deployment & MLOps	Streamlit UI, model/version tracking, logging, reproducible inference, basic monitoring.
Career Capstone	GitHub-quality documentation, architecture diagram, tests, demo, presentation, and technical defense.

How to Choose

- Each student/team chooses ONE project from the 10 options below.
- The selected project must contain a classical ML baseline AND a Deep Learning upgrade whenever the problem naturally supports it.
- Teams are expected to compare models rather than present one model only.
- The final application must expose predictions/results through a usable interface; Streamlit is the recommended UI.
- FastAPI is optional for students who want a backend API, but it is not mandatory.
- No project is considered complete if it only consists of a notebook and a high accuracy number.

Project 1: AI Credit Risk & Loan Default Intelligence Platform

1. Business / Real-World Problem

Build a decision-support platform for a lending institution. Given an applicant and their historical financial information, estimate default probability, rank risk, explain the strongest drivers, and provide a decision-support report. The system is not a replacement for a human credit officer.

2. Dataset & Data Sources

- **Home Credit Default Risk** — Kaggle: [Open dataset / source](#) — Multi-table credit data suitable for joins, feature engineering, risk modeling and SQL.
- **UCI Adult** — optional fairness/feature-engineering extension: [Open dataset / source](#) — Optional secondary dataset for classification/fairness experiments.

3. Core Objectives

- Create a clean analytical data model from the multiple Home Credit tables.
- Perform advanced EDA: missingness, distributions, leakage checks, target imbalance and relationship analysis.
- Engineer aggregate features from previous credit/application history.
- Build a reproducible ML pipeline from raw data to prediction.
- Compare interpretable and high-performance models.
- Add probability calibration so the output is a meaningful risk score.
- Create an explanation layer showing the top factors affecting each prediction.

4. Machine Learning Track — Required

1. Build a baseline Logistic Regression.
2. Compare Decision Tree, Random Forest and Gradient Boosting/XGBoost-style model.
3. Use stratified cross-validation and a robust preprocessing pipeline.
4. Handle class imbalance using class weights and/or carefully justified resampling.
5. Optimize threshold based on business cost, not accuracy alone.
6. Report ROC-AUC, PR-AUC, recall, precision, F1, confusion matrix and calibration.
7. Perform feature importance and error analysis by risk segment.

5. Deep Learning Track — Required Upgrade

8. Build an MLP for tabular classification using normalized numerical features and encoded categorical features.
9. Compare MLP performance against the best classical model.
10. Use early stopping, learning-rate scheduling and regularization.
11. Perform probability calibration for the DL model as well.
12. Optional: build an autoencoder-based anomaly score and combine it with supervised risk.

6. Advanced / Distinction Features

- SHAP-based local and global explanations.
- Fairness analysis across selected demographic/proxy groups where ethically appropriate.
- Cost-sensitive decision simulation: false negative vs false positive cost.
- Model registry with version, training data hash and evaluation metrics.
- Champion/challenger model comparison.
- Drift simulation by changing the distribution of selected input variables.

7. Deployment & MLOps

- Streamlit applicant-risk dashboard.
- Save preprocessing + model pipeline as a single reproducible artifact.
- Log prediction timestamp, model version and input schema version.
- Containerization is optional; a simple reproducible environment file is required.
- Deploy the demo to a cloud-friendly platform such as Hugging Face Spaces/Streamlit-compatible hosting.

9. Required Final Deliverables

- GitHub repository with README and architecture diagram.
- Data dictionary and SQL schema/queries.
- EDA notebook/report.
- ML training notebook or scripts.
- DL training notebook or scripts.
- Model comparison table.
- Streamlit application.
- Explainability report.
- Final technical report (10–20 pages).
- 5–10 minute demo + technical defense.

10. Evaluation & Success Criteria

- Primary metrics: PR-AUC, ROC-AUC, recall, precision, F1, calibration/Brier score.
- Target is not a universal numeric threshold; teams must justify their chosen threshold using a business cost matrix.
- DL model should demonstrate a meaningful experiment, not merely be added for decoration.

11. Common Risks / Quality Checks

- Prevent target leakage from post-loan information.
- Do not use accuracy as the main metric under strong class imbalance.
- Document all assumptions around sensitive attributes.
- Do not claim the model is a real-world lending approval system.

Project 2: Real-Time Credit Card Fraud Detection & Anomaly Intelligence

1. Business / Real-World Problem

Build a fraud engine that scores incoming transactions and identifies suspicious behavior. The project must explicitly address the fact that fraud is a rare event and that false negatives can be much more expensive than false positives.

2. Dataset & Data Sources

- **Credit Card Fraud Detection — Kaggle / ULB-MLG:** [Open dataset / source](#) — 284,807 transactions with 492 frauds; highly imbalanced classification.
- **Fraud Detection Handbook:** [Open dataset / source](#) — Optional reference/simulator for more realistic temporal fraud experiments.

3. Core Objectives

- Explore temporal and amount patterns.
- Create a leakage-safe train/validation/test strategy.
- Build supervised fraud classifiers.
- Add unsupervised anomaly detection.
- Create a transaction-risk score and decision threshold.
- Simulate a stream of incoming transactions.
- Build monitoring dashboards for fraud rate, alerts and model performance.

4. Machine Learning Track — Required

13. Logistic Regression baseline.
14. Random Forest and gradient boosting comparison.
15. Class weighting and/or under-sampling experiment.
16. PR-AUC as a primary evaluation metric.
17. Threshold tuning using an explicit fraud-loss matrix.
18. Temporal validation: avoid random splitting when testing streaming behavior.
19. Error analysis of false negatives and false positives.

5. Deep Learning Track — Required Upgrade

20. MLP classifier for fraud probability.
21. Autoencoder trained on normal transactions for anomaly detection.
22. Compare supervised DL, autoencoder anomaly score and classical ML.
23. Optional hybrid score: supervised probability + anomaly score.
24. Stress-test the DL model under changing fraud rates.

6. Advanced / Distinction Features

- Streaming simulation with batches.
- Concept-drift experiment.
- Cost-sensitive alert prioritization.
- Model monitoring page.
- Champion/challenger models.
- Optional Spark Structured Streaming extension.

7. Deployment & MLOps

- Streamlit transaction simulator and fraud dashboard.
- Batch scoring script that accepts CSV files.
- Prediction logging.
- Model/version metadata.
- Optional Spark pipeline; Apache Spark MLlib is suitable for scalable ML pipelines and collaborative filtering.

9. Required Final Deliverables

- Complete data pipeline.
- ML + DL comparison.
- Anomaly detector.
- Cost matrix.
- Streamlit dashboard.
- Monitoring report.
- Model card and limitations.
- GitHub repository + final report + demo.

10. Evaluation & Success Criteria

- PR-AUC, recall at a fixed precision, precision, F1, false-negative rate and business cost.
- Report alert volume so the solution is operationally realistic.

11. Common Risks / Quality Checks

- Do not oversell anonymized PCA features as interpretable business features.
- Do not randomly split data if the experiment claims real-time deployment behavior.
- Do not use accuracy as the headline metric.

Project 3: Retail Customer 360, Segmentation & Recommendation Engine

1. Business / Real-World Problem

Build an end-to-end retail intelligence platform that understands customers, products and purchasing behavior. The system should answer: Who are the valuable customers? Who is likely to churn? What should we recommend next?

2. Dataset & Data Sources

- **UCI Online Retail II:** [Open dataset / source](#) — 1M+ transactions across two years; classification, regression and clustering tasks are possible.
- **MovieLens 32M** — **optional recommendation benchmark:** [Open dataset / source](#) — Optional second dataset if the team wants a pure recommender-system benchmark.

3. Core Objectives

- Create a clean transaction warehouse/schema.
- Use SQL for customer/product aggregation and analytical queries.
- Build RFM and behavioral features.
- Perform customer segmentation.
- Predict customer churn or repeat purchase.
- Build a recommendation engine.
- Expose insights through a dashboard.

4. Machine Learning Track — Required

25. K-Means or Gaussian Mixture segmentation.
26. Logistic Regression / Random Forest / Gradient Boosting for repeat-purchase or churn prediction.
27. Association-rule baseline for product relationships.
28. Collaborative filtering baseline using implicit/explicit interactions.
29. Cross-validation and business-oriented evaluation.

5. Deep Learning Track — Required Upgrade

30. Build an MLP for customer churn/repeat-purchase prediction.
31. Optional neural collaborative filtering or embedding-based recommendation model.
32. Optional sequence model for next-item prediction using purchase history.
33. Compare neural recommendations against a classical collaborative-filtering baseline.

6. Advanced / Distinction Features

- Customer lifetime value estimation.
- Market-basket analysis.
- Personalized promotion simulator.
- Segment-level dashboard.
- Cold-start strategy.
- Recommendation diversity and novelty analysis.

7. Deployment & MLOps

- Streamlit Customer 360 dashboard.
- Scheduled feature-generation script.
- Persist model + feature schema.
- Optional Spark implementation for transaction aggregation.
- SQL database containing customers, products, orders, recommendations and prediction logs.

9. Required Final Deliverables

- SQL schema + 10+ analytical queries.
- RFM notebook/report.
- Segmentation model.
- Churn/repeat-purchase model.
- Recommendation engine.
- DL upgrade.
- Streamlit Customer 360.
- Advanced AI analyst.
- Final report and presentation.

10. Evaluation & Success Criteria

- Clustering: silhouette score + business interpretability.
- Classification: PR-AUC/ROC-AUC/F1 as appropriate.
- Recommendation: Precision@K, Recall@K, MAP@K or NDCG@K.
- Business metrics: expected revenue uplift in a simulation.

11. Common Risks / Quality Checks

- Handle returns/cancellations carefully.
- Avoid using future transactions to create past features.
- Separate offline recommendation evaluation from claims about real customer impact.

Project 4: Smart Energy Forecasting & Consumption Optimization

1. Business / Real-World Problem

Build a time-series forecasting system for household/building energy consumption. The solution should forecast future energy demand and identify abnormal or inefficient consumption patterns.

2. Dataset & Data Sources

- **UCI Appliances Energy Prediction:** [Open dataset / source](#) — 19,735 observations, 10-minute sampling, environmental and energy variables.

3. Core Objectives

- Build a leakage-safe time-series pipeline.
- Create lag, rolling and calendar features.
- Perform exploratory time-series analysis.
- Establish naive and classical forecasting baselines.
- Train ML regressors.
- Train a sequence model.
- Convert forecasts into a simple optimization/recommendation layer.

4. Machine Learning Track — Required

34. Naive persistence baseline.
35. Linear Regression / Random Forest / Gradient Boosting with lag features.
36. TimeSeriesSplit validation.
37. Feature importance and residual analysis.
38. Compare different forecast horizons.

5. Deep Learning Track — Required Upgrade

39. LSTM or GRU for sequence forecasting.
40. Optional 1D CNN + LSTM hybrid.
41. Use early stopping and learning-rate scheduling.
42. Compare deep sequence model against feature-based ML.
43. Optional uncertainty estimation via ensembles or quantile-style prediction.

6. Advanced / Distinction Features

- Anomaly detection on residuals.
- Peak-demand detection.
- Energy-saving recommendation rules.
- Forecast dashboard with confidence/uncertainty display.
- Optional Spark processing for large synthetic expansion.

7. Deployment & MLOps

- Streamlit forecast dashboard.
- Upload a CSV and receive a forecast.
- Store forecasts and actuals for monitoring.
- Track MAE/RMSE over time.
- Optional scheduled batch forecasting.

9. Required Final Deliverables

- Time-series data dictionary.
- Feature-engineering pipeline.
- ML baseline comparison.
- LSTM/GRU model.
- Forecast visualization.
- Residual/anomaly analysis.
- Streamlit application.
- Advanced AI advisor.
- Final report.

10. Evaluation & Success Criteria

- MAE, RMSE and optionally MAPE/SMAPE where mathematically appropriate.
- Report results separately for normal periods and peak-demand periods.

11. Common Risks / Quality Checks

- Never shuffle time-series observations for final evaluation.
- Prevent future information leakage through rolling/lag features.
- Do not compare models using only training loss.

Project 5: Industrial Predictive Maintenance & Failure Prevention

1. Business / Real-World Problem

Build an industrial monitoring platform that estimates failure risk and recommends maintenance actions. The project must treat false negatives as costly and should include model interpretability.

2. Dataset & Data Sources

- **AI4I 2020 Predictive Maintenance — UCI:** [Open dataset / source](#) — Synthetic industrial process data with machine-failure targets and multiple failure modes.
- **UCI Time-Series datasets — optional extensions:** [Open dataset / source](#) — Optional source for a more temporal experiment.

3. Core Objectives

- Understand machine operating variables and failure labels.
- Build a predictive-maintenance feature pipeline.
- Compare classification models.
- Investigate class imbalance.
- Add anomaly detection.
- Create maintenance-risk tiers.
- Simulate incoming sensor records.

4. Machine Learning Track — Required

44. Logistic Regression baseline.
45. Decision Tree / Random Forest.
46. Gradient Boosting.
47. Class-weighting experiment.
48. Threshold optimization.
49. PR-AUC/recall-focused evaluation.
50. Explainability with feature importance/SHAP.

5. Deep Learning Track — Required Upgrade

51. MLP for tabular sensor data.
52. Autoencoder for unsupervised anomaly detection.
53. Optional LSTM/GRU if a real sequential dataset is added.
54. Compare supervised failure prediction with anomaly score.

6. Advanced / Distinction Features

- Failure-mode classification.
- Remaining useful life extension if a suitable dataset is added.
- Drift detection.
- Maintenance scheduling simulator.
- Cost-based alerting.

7. Deployment & MLOps

- Streamlit machine-health dashboard.
- Batch/stream scoring.
- Prediction history database.
- Monitoring for feature drift and alert rate.
- Optional Spark implementation for large sensor simulations.

9. Required Final Deliverables

- EDA + data dictionary.
- SQL or structured storage.
- ML benchmark.
- DL/anomaly model.
- Explainability.
- Monitoring.
- Streamlit app.
- RAG/agent.
- Final technical report and demo.

10. Evaluation & Success Criteria

- PR-AUC, recall, precision, F1, false-negative rate.
- Business simulation: estimated cost of missed failures vs unnecessary maintenance.

11. Common Risks / Quality Checks

- Do not claim real industrial reliability from a synthetic benchmark.
- Document assumptions about sensor frequency and failure timing.
- Do not leak failure-related variables from the future.

Project 6: Malaria Cell Detection & Explainable Computer Vision

1. Business / Real-World Problem

Create a computer-vision screening prototype for educational/research use. The goal is not clinical diagnosis; it is to demonstrate a rigorous image-classification workflow, comparison of classical ML and DL, and explainability.

2. Dataset & Data Sources

- **Malaria Cell Images Dataset — Kaggle:** [Open dataset / source](#) — 27,558 cell images in infected/uninfected folders; source acknowledgement points to NIH.
- **NIH Malaria Datasets:** [Open dataset / source](#) — Primary source reference for the dataset family.

3. Core Objectives

- Inspect class balance and image quality.
- Create reproducible train/validation/test splits.
- Perform image preprocessing and augmentation.
- Build a classical ML image baseline using handcrafted features.
- Build a CNN/transfer-learning model.
- Perform error analysis by image characteristics.
- Build an image-upload application.

4. Machine Learning Track — Required

55. Baseline: resize images + HOG/color/texture features + SVM or Random Forest.
56. Evaluate classical feature-based pipeline.
57. Build a confusion matrix and per-class metrics.
58. Document the limits of handcrafted features.

5. Deep Learning Track — Required Upgrade

59. Custom CNN baseline.
60. Transfer learning with a modern pretrained CNN such as EfficientNet/ResNet.
61. Data augmentation.
62. Class weights if needed.
63. Early stopping and learning-rate scheduling.
64. Compare custom CNN vs transfer learning.
65. Grad-CAM or similar visual explanation.

6. Advanced / Distinction Features

- Test-time augmentation.
- Calibration of predicted probabilities.
- Robustness testing under blur/brightness/rotation.
- Optional ensemble of two CNN models.
- Optional segmentation/preprocessing extension if a suitable mask dataset is added.

7. Deployment & MLOps

- Streamlit image upload UI.
- Display predicted class + confidence + explanation heatmap.
- Store inference logs without storing sensitive user information.
- Version model and preprocessing pipeline.

9. Required Final Deliverables

- Data preparation notebook.
- Classical ML baseline.
- CNN model.
- Transfer-learning model.
- Grad-CAM visualizations.
- Robustness tests.
- Streamlit demo.
- Final report with responsible-AI section.

10. Evaluation & Success Criteria

- Accuracy is insufficient alone.
- Report precision, recall, F1, ROC-AUC, PR-AUC and confusion matrix.
- Focus on sensitivity/recall for the infected class while explaining trade-offs.

11. Common Risks / Quality Checks

- Do not present the system as clinical diagnosis.
- Check for data leakage caused by duplicated/near-duplicated images.
- Document dataset license and provenance.

Project 7: AI Waste Segregation & Smart Recycling Assistant

1. Business / Real-World Problem

Build a practical waste-management vision system. A user uploads an image of an item; the system classifies it, shows confidence, and recommends the disposal category. The project should demonstrate computer vision, deployment and an explainable user experience.

2. Dataset & Data Sources

- **Garbage Dataset v2 — Kaggle:** [Open dataset / source](#) — Multi-class garbage images designed for classification/recycling applications.

3. Core Objectives

- Inspect class balance and visual diversity.
- Create robust image preprocessing.
- Build a classical feature-based baseline.
- Build CNN and transfer-learning models.
- Add confidence thresholding and an 'uncertain' state.
- Build a user-facing application.
- Add a recycling knowledge layer.

4. Machine Learning Track — Required

66. HOG + SVM or another classical image-feature baseline.
67. Compare classical feature engineering against DL.
68. Use class-wise precision/recall/F1.
69. Analyze confusion between visually similar categories.

5. Deep Learning Track — Required Upgrade

70. Custom CNN.
71. Transfer learning with ResNet/EfficientNet/MobileNet.
72. Data augmentation.
73. Fine-tuning of upper layers.
74. Grad-CAM or saliency explanation.
75. Optional lightweight model comparison for mobile deployment.

6. Advanced / Distinction Features

- Unknown/other rejection class.
- Image-quality detector.
- Latency vs accuracy benchmark.
- Quantization/size comparison.
- Optional object detection extension for multiple waste items.

7. Deployment & MLOps

- Streamlit image upload app.
- Prediction history.
- Model confidence + explanation.
- Optional deployment to Hugging Face Spaces.
- Model artifact + requirements + reproducible inference script.

9. Required Final Deliverables

- Dataset analysis.
- Classical baseline.
- CNN.
- Transfer learning.
- Explainability.
- Deployment.
- Recycling RAG.
- Final report and presentation.

10. Evaluation & Success Criteria

- Macro F1 and per-class metrics.
- Top-1 accuracy and confusion matrix.
- Latency and model size for deployment.
- Human-readable uncertainty behavior.

11. Common Risks / Quality Checks

- Dataset images may be cleaner than real-world photos; test robustness with student-collected images.
- Do not imply recycling rules are universal; document the geographic scope of the guidance.

Project 8: Intelligent Traffic Sign Recognition System

1. Business / Real-World Problem

Develop a traffic-sign recognition prototype. The project should demonstrate a complete computer-vision pipeline from image preprocessing and classical ML to CNN/transfer learning and deployment.

2. Dataset & Data Sources

- **German Traffic Sign Recognition Benchmark (GTSRB)** — official: [Open dataset / source](#) — Official benchmark for multi-category traffic-sign recognition.

3. Core Objectives

- Explore all sign classes and class balance.
- Build preprocessing and augmentation.
- Create a handcrafted-feature baseline.
- Train CNN models.
- Analyze visually similar classes.
- Build real-time-ish inference through image upload/webcam where feasible.

4. Machine Learning Track — Required

76. HOG + SVM baseline.
77. Random Forest as an alternative feature baseline.
78. Per-class confusion matrix.
79. Threshold and top-k prediction analysis.

5. Deep Learning Track — Required Upgrade

80. Custom CNN.
81. Transfer learning with ResNet/MobileNet/EfficientNet.
82. Augmentation for rotation, brightness and scale.
83. Fine-tuning.
84. Top-1 and Top-5 evaluation.
85. Grad-CAM for explanation.

6. Advanced / Distinction Features

- Robustness under blur, illumination and perspective changes.
- Model compression/latency benchmark.
- Optional object-detection extension if bounding-box data is added.
- Optional video-frame smoothing.

7. Deployment & MLOps

- Streamlit upload + webcam-capable prototype where supported.
- Top-3 predictions.
- Latency measurement.
- Model versioning.
- Optional cloud deployment.

9. Required Final Deliverables

- Data preparation.
- Classical baseline.
- CNN.
- Transfer learning.
- Robustness tests.
- Explainability.
- Deployment.
- Final report.

10. Evaluation & Success Criteria

- Accuracy, macro F1, per-class recall and confusion matrix.
- Report performance under transformed/perturbed images.

11. Common Risks / Quality Checks

- Dataset domain is German traffic signs; document this limitation.
- Do not claim road-safety certification.

Project 9: Customer Sentiment & Emotion Intelligence Platform

1. Business / Real-World Problem

Build an NLP system that classifies emotion/sentiment and turns raw customer text into actionable support intelligence. The platform should compare classical NLP against deep learning and then add a grounded Advanced AI assistant.

2. Dataset & Data Sources

- **Emotion Dataset — Hugging Face / DAIR.AI:** [Open dataset / source](#) — English text classification dataset with six emotion classes.
- **Amazon Reviews Multi — Hugging Face:** [Open dataset / source](#) — Optional large multilingual review dataset for stronger NLP experiments.

3. Core Objectives

- Text cleaning and exploratory analysis.
- TF-IDF feature extraction.
- Classical text classification.
- Error analysis by emotion.
- Deep sequence/transformer model.
- Batch inference and dashboard.
- Generate aggregated customer-insight reports.

4. Machine Learning Track — Required

86. TF-IDF + Logistic Regression.
87. TF-IDF + Linear SVM.
88. Compare n-gram configurations.
89. Class-wise precision/recall/F1.
90. Calibration and threshold analysis.

5. Deep Learning Track — Required Upgrade

91. Embedding-based MLP or CNN/RNN baseline.
92. LSTM/GRU sequence classifier.
93. Optional transformer fine-tuning using a suitable pretrained model.
94. Compare classical NLP vs LSTM vs transformer.
95. Use early stopping and learning-rate scheduling.

6. Advanced / Distinction Features

- Topic discovery for complaint themes.
- Emotion + topic matrix.
- Priority score combining emotion, topic and business rules.
- Multilingual extension using Amazon Reviews Multi.
- Human review sampling.

7. Deployment & MLOps

- Streamlit text/file upload.
- Batch CSV classification.
- Charts for emotion trends.
- Model/version logging.
- Optional deployment.

9. Required Final Deliverables

- Preprocessing notebook.
- TF-IDF baseline.
- LSTM/GRU or transformer.
- Model comparison.
- Topic/emotion dashboard.
- RAG assistant.
- Streamlit app.
- Final report.

10. Evaluation & Success Criteria

- Macro F1, weighted F1, per-class recall.
- For the assistant: factuality/groundedness evaluation using a manually labeled sample.
- Report confusion between semantically close emotions.

11. Common Risks / Quality Checks

- Do not treat emotion labels as clinical mental-health assessments.
- Check class imbalance.
- Document model bias and limitations.

Project 10: Enterprise AI Data Analyst: SQL + Spark + RAG + Advanced AI Decision Support

1. Business / Real-World Problem

Create an enterprise-style AI analyst for an online retail company. A manager should be able to ask business questions, view KPIs, inspect ML predictions, and receive grounded explanations. The system must combine deterministic analytics with Advanced AI rather than letting an LLM invent numbers.

2. Dataset & Data Sources

- **UCI Online Retail II:** [Open dataset / source](#) — Large transaction dataset ideal for SQL, ETL, clustering, forecasting and business analytics.
- **Apache Spark MLlib documentation:** [Open dataset / source](#) — Use as the technical reference for scalable Spark ML pipelines.

3. Core Objectives

- Create a normalized SQL data model.
- Build ETL/ELT pipeline from raw transactions.
- Use advanced SQL: joins, CTEs, window functions and aggregations.
- Build customer/product features.
- Train at least one predictive ML model.
- Add one DL model where meaningful, such as demand forecasting or customer prediction.
- Use Spark for scalable transformation or ML.
- Build a RAG knowledge layer.
- Build a controlled analytics agent.
- Deploy the complete application.

4. Machine Learning Track — Required

96. Customer segmentation or repeat-purchase prediction.
97. Demand/revenue forecasting using feature-based ML.
98. Model selection and tuning.
99. Cross-validation or time-aware validation depending on task.
100. Explainability.
101. Business KPI simulation.

5. Deep Learning Track — Required Upgrade

102. Choose one DL component: LSTM/GRU forecasting, MLP customer prediction, or neural recommendation.
103. Compare against the classical ML baseline.
104. Track experiment configuration and metrics.
105. Optional transformer-based text component if product descriptions/reviews are added.

6. Advanced / Distinction Features

- Spark ETL pipeline.
- Feature store-style table.
- Read-only SQL execution layer.
- Agent tool routing.
- RAG citations.
- Query/result caching.
- Prompt-injection defenses.
- Role-based access concept.
- Audit logs for agent actions.

7. Deployment & MLOps

- Streamlit executive dashboard.
- SQL database.
- Spark pipeline.
- Model artifacts.
- Prediction logging.
- Read-only analytics tool.
- Cloud-ready architecture.
- Optional Docker.
- Optional FastAPI backend.

9. Required Final Deliverables

- Architecture diagram.
- SQL schema.
- ETL code.
- Spark job.
- ML model.
- DL model.
- RAG pipeline.
- Agent workflow.
- Streamlit application.
- Tests.
- Security/guardrail document.
- Final technical report and live demo.

10. Evaluation & Success Criteria

- Analytics correctness: compare generated answers with ground-truth SQL queries.
- ML metrics appropriate to the chosen task.
- Agent metrics: tool-selection accuracy, groundedness, hallucination rate and task completion rate.
- Performance metrics: query latency and inference latency.

11. Common Risks / Quality Checks

- Do not permit unrestricted LLM-generated SQL against a production database.
- Prevent prompt injection in retrieved documents.
- Separate deterministic calculations from generated natural language.
- Keep secrets/API keys outside source control.



Universal Graduation Project Requirements

Regardless of the selected project, the following checklist is mandatory for a project to be accepted as a full graduation project. Every project must also include a professional GitHub repository and a Dockerized application or reproducible Docker setup.

Area	Mandatory Requirement	Evidence
Data Engineering	Raw → cleaned → feature-ready pipeline.	Scripts/notebooks + data dictionary.
SQL	At least 10 meaningful analytical queries; include JOIN, CTE and window function where applicable.	SQL file + screenshots/results.
EDA	Univariate, bivariate and target-focused analysis.	EDA report.
ML	At least 3 meaningful ML models or baselines.	Comparison table + saved metrics.
Validation	Leakage-safe validation strategy.	Documented split strategy.
Tuning	Hyperparameter search for at least the best ML model.	Search results.
Metrics	Use task-appropriate metrics; do not rely on accuracy only.	Evaluation report.
DL	At least one meaningful deep-learning model.	Training curves + test results.
Comparison	Classical ML vs DL comparison.	Final comparison table.
Explainability	Feature importance/SHAP or Grad-CAM/text explanation as appropriate.	Explanation examples.
Deployment	Working Streamlit interface.	Live/local demo.
MLOps	Versioned model + reproducible requirements + prediction logging.	Repository structure.
Testing	At least basic data/model/UI tests.	Test folder/report.
Documentation	Professional README and architecture diagram.	GitHub repository.
Presentation	Technical defense, not only product demo.	Slides + live Q&A.
Docker	Containerize the Streamlit application and provide a reproducible Docker build/run process.	Dockerfile + requirements + run instructions.
GitHub	Publish a clean repository with README, project structure, setup instructions and Git history.	GitHub repository link + README.

Suggested Repository Structure

```

project-name/
├── data/                # raw/processed metadata only; do not upload huge datasets
├── notebooks/          # EDA + experiments
├── src/
│   ├── data/           # loading + validation
│   ├── features/        # feature engineering
│   ├── models/          # training + inference
│   ├── evaluation/      # metrics + error analysis
│   └── utils/
├── sql/                # schema + analytical queries
├── spark/              # Spark jobs where applicable
├── rag/                # ingestion + retrieval + prompts
├── app/                # Streamlit application
├── tests/              # unit/integration tests
├── models/             # lightweight/versioned artifacts
├── reports/            # figures + final report
├── requirements.txt
├── .env.example
└── README.md
  
```

Final Report Structure

106.Executive Summary
 107.Problem Definition and Business Context
 108.Dataset Description and Data Governance
 109.System Architecture
 110.SQL/Data Pipeline Design
 111.EDA and Statistical Analysis
 112.Feature Engineering
 113.Machine Learning Experiments
 114.Deep Learning Experiments
 115.Model Evaluation and Error Analysis
 116.Explainability / Responsible AI
 117.Deployment Architecture
 118.MLOps and Monitoring
 119.Advanced AI Integration
 120.Testing and Security
 121.Limitations
 122.Future Work
 123.Conclusion
 124.References

Final Presentation Checklist

- Problem → why it matters → users.
- Dataset → size → variables → target → limitations.
- Architecture diagram.
- SQL/data pipeline demo.
- EDA insight examples.
- ML baseline comparison.
- DL upgrade and why it helps.
- Evaluation and error analysis.
- Live application demo.
- AI assistant / advanced integration demonstration.
- Failure cases and limitations.
- What would be required to take the project to production?

Recommended Project Ranking

Rank	Project	Why it is strong
1	Enterprise AI Data Analyst	Most complete coverage of SQL + Spark + ML + DL + RAG + agents + deployment.
2	Credit Risk Intelligence	Excellent tabular ML, explainability, imbalance, SQL and business decision-making.
4	Predictive Maintenance	Strong industrial story with anomaly detection, DL and monitoring.
5	Malaria Cell Vision	Strong computer-vision progression from classical ML to CNN/transfer learning.
6	Waste Segregation	Excellent practical CV demo with an easy-to-understand product.
7	Traffic Sign Recognition	Good CV benchmark with clear model comparison and deployment.
8	Fraud Detection	Very strong ML problem; excellent for imbalanced learning and anomaly detection.
9	Energy Forecasting	Strong time-series and LSTM project with useful visualization.
10	Customer Emotion Intelligence	Strong NLP progression from TF-IDF to LSTM/Transformer + RAG.

Dataset Source Notes

Dataset links were selected from current public dataset pages and official/reputable repositories. Students must read the license/terms of each dataset before redistribution or publication. For Kaggle datasets, downloading may require a Kaggle account. The project report must cite the original dataset creators.

- **UCI Adult:** <https://uci-ics-mlr-prod.aws.uci.edu/dataset/2/adult>
- **Kaggle Home Credit Default Risk:** <https://www.kaggle.com/competitions/home-credit-default-risk/data>
- **Kaggle Credit Card Fraud Detection:** <https://www.kaggle.com/mlg-ulb/creditcardfraud>
- **UCI Online Retail II:** <https://archive.ics.uci.edu/dataset/502/online%2Bretail%2Bii>
- **GroupLens MovieLens 32M:** <https://grouplens.org/datasets/movielens/32m/>
- **UCI Appliances Energy Prediction:**
<https://archive.ics.uci.edu/dataset/374/appliances%2Benergy%2Bprediction>
- **UCI AI4I Predictive Maintenance:**
<https://archive.ics.uci.edu/dataset/601/ai4i%2B2020%2Bpredictive%2Bmaint>
- **Kaggle Malaria Cell Images:** <https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria/data>
- **NIH Malaria Datasets:** <https://ceb.nlm.nih.gov/repositories/malaria-datasets/>
- **Kaggle Garbage Dataset:** <https://www.kaggle.com/datasets/sumn2u/garbage-classification-v2/data>
- **Official GTSRB:** https://benchmark.ini.rub.de/gtsrb_dataset.html
- **Hugging Face Emotion:** <https://huggingface.co/datasets/dair-ai/emotion>
- **Hugging Face Amazon Reviews Multi:** https://huggingface.co/datasets/amazon_reviews_multi
- **Apache Spark MLlib Guide:** <https://spark.apache.org/docs/latest/ml-guide.html>

Simplified Graduation Project Track: Machine Learning → Deep Learning → Deployment → Docker → GitHub